🧭 HANDOFF BRIEFING · UPDATED 2026-08-25

ACE: certifying agents under real Kubernetes chaos

ACE (Agent Certification Engine) injects real infrastructure faults into a live Kubernetes cluster, lets an AI agent attempt autonomous remediation, and runs its full LLM trace through a 4-phase pipeline to produce a statistical certification report. This page orients a new developer picking up the project: what exists, how it fits together, what was just fixed, and what's still open.

AGENTS ONBOARDED	ACTIVELY TESTED RECENTLY	MODEL ROUTING
6 charts — flash-agent, ciso-agent, sre-agent, sre-agent-comprehensive, sre-agent-crewai, k8s-agent	sre-agent-comprehensive only — 29/29 ITBench faults via live UI trigger	Ollama qwen2.5 (local GPU) + Azure `gpt-4o` alias
GPU	**LEGACY DATASET**	**BRANCH**
RTX A6000, 49GB — ~40–100× vs CPU	137/137 SRE runs — flash-agent + local Ollama, 2026-08-12	feature/itbench-scenarios

⚠ Read this first — three corrections to how this project has been reported

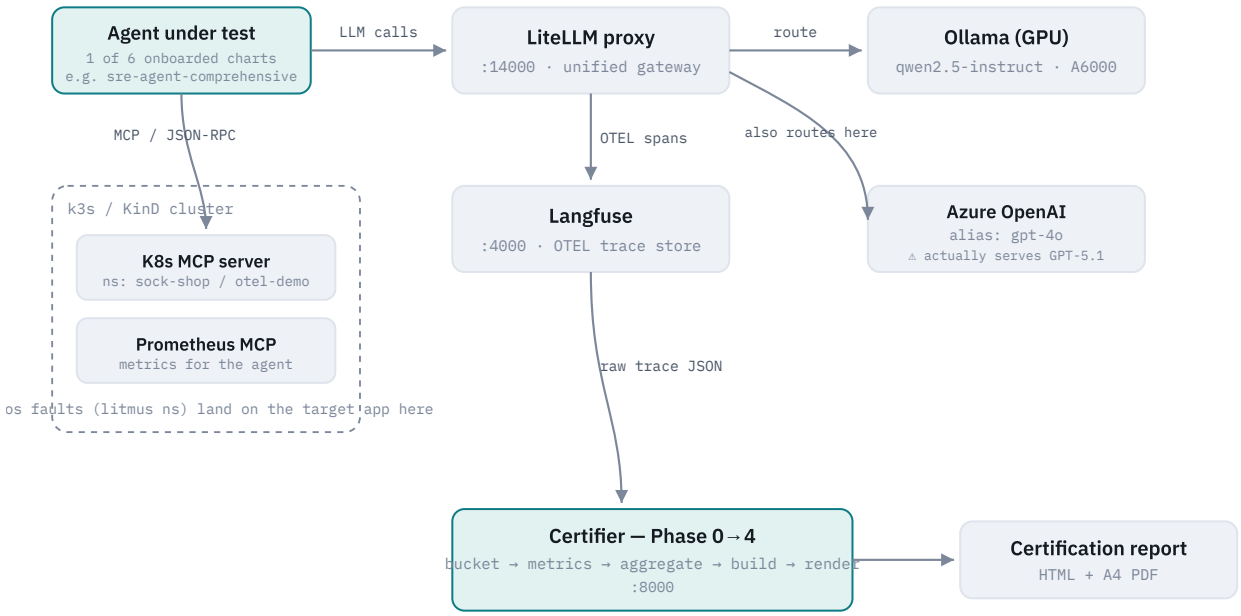
1. **The `gpt-4o` alias does not actually serve GPT-4o.** Azure's own `x-ms-served-model` response header confirms this environment's `gpt-4o` deployment is silently GPT-5.1, a reasoning-class model with different param requirements (`max_completion_tokens` , no `stop`). Any past result labeled "gpt-4o" was produced against this same backend without anyone knowing it — see today's fixes and model-identity gap.
2. **The 137/137 SRE figure is a historical baseline, not current data.** It's flash-agent, on local Ollama, from 2026-08-12 — before the Azure routing above existed and before the platform's real UI-trigger path was exercised. The current, actively-tested agent is `sre-agent-comprehensive` ; see its most recent full run.

3. **Commit status is disputed.** This session was told everything from today is committed. A direct `git status` in this checkout still shows the same uncommitted diffs described below (`setup.sh`, both handoff docs, the `AgentCert` and `chaos-charts` submodules). Flagged rather than asserted either way — see Current problems.
4. **The rootless-Docker migration produced more than the two gaps this page used to mention.** Six distinct incidents, one of them a still-unfixed per-user kernel resource exhaustion that silently blocks new pod creation cluster-wide — see the dedicated Rootless Docker section below.

01 · SYSTEM MAP

How a fault becomes a certificate

One end-to-end run: an onboarded agent (there are 6 charts on the platform — this is not flash-agent-specific, though flash-agent uses MCP over SSE while the CrewAI-based agents use MCP streamable-HTTP with a fresh session per call) operates against a live cluster while a fault is injected, every LLM call it makes is traced, and once the run ends the trace is fed through certifier's four phases to produce a scored report.



agent-sidecar stamps EXPERIMENT_ID / RUN_ID on every LLM call · LiteLLM currently load-balances between local Ollama and the Azure gpt-4o alias · every certificate is later traceable back through this same trace

02 · CERTIFIER/

The four phases that turn a trace into a score

LLMs classify and observe; **Python does all the arithmetic** — TTD/TTR, hallucination score, success rate. Phase 3's Pydantic validation is a hard gate: a malformed report raises instead of shipping.

chips outlined in accent = at least one LLM call in that phase; plain = deterministic Python

Phase 3's Pydantic v2 validation fails hard rather than emit a malformed certificate

03 · PROGRESS LOG, CONDENSED

What's been done, in order

The full change log lives in `OPEN_WEIGHT_CERTIFICATION_HANDOFF.md` — 80 numbered entries with commit SHAs, exact commands, and verification steps. This is the shape of it.

30 ITBench SRE fault bundles completed

• baseline

All 30 ITBench scenarios landed as LitmusChaos ChaosHub fault bundles under `chaos-charts/faults/itbench/`, before this certification effort began.

CISO scorecard pipeline stood up

Metrics adapter + CISO-specific aggregation functions added to certifier, behind an opt-in `--include-ciso-finops` flag.

GPU discovered mid-session

• unblocked everything

An idle RTX A6000 (49GB) was found on the host with drivers already installed. Ollama picked it up with zero config changes — ~40–100× speedup, turning a multi-day mass-execution plan into a same-session one.

Mass execution: 137/137 SRE runs

• historical baseline

29 fault bundles × 5 runs (2 high-blast-radius faults capped at 1) — **flash-agent, local Ollama, 2026-08-12.** A cleanup-timing bug in the run driver (deleting the fault before it finished reverting) caused real cluster damage that was found and fully repaired. This predates the Azure gpt-4o routing and the platform's real UI-trigger path — treat it as a proof-of-concept dataset, not the current state of certification coverage.

First real SRE certification report

Full Phase 0→4 run producing a 20-page PDF with real narrative content — surfaced 7 latent report-rendering bugs that had never been hit because Phase 3/4 had never run end-to-end before.

CISO agent LLM routing fixed, first trial PASS

Two separate bugs in `ciso-agent/src/ciso_agent/llm.py` (LangChain path and CrewAI/litellm path each had their own hardcoded assumption that the model was a GPT variant) blocked any open-weight model from running the CISO agent at all.

CISO mass execution + first CISO certification report

5 runs across 3 scenario types; two genuine small-model reliability failures recorded honestly as data, not hidden. First-ever CISO PDF produced through Phase 2→4.

Submodule fork drift corrected

`chaos-charts`, `app-charts`, and `certifier` had silently drifted to point at a contributor's personal fork instead of the `AgentCert` org — meaning content could exist locally yet be invisible to the deployed ChaosHub. Repointed and made this a standing rule in `CLAUDE.md` §1.

Post-certification portability hardening `2026-08-05 → 08-11`

Swept every submodule for hardcoded Docker-bridge IPs and service names that were never real (`litellm-proxy.litellm` vs. the actual `litellm.ace.svc.cluster.local`) — none of these had ever produced an error message, they just silently never worked for anyone else cloning fresh.

Rootless-Docker rework + MongoDB replica-set fix `2026-08-12 →`

Bridge networking for `auth` / `graphql` / `web`, a KinD internal-kubeconfig bridge so `graphql` can reach the API server without host networking, personal CDI GPU passthrough, and a Kubernetes replica-set bug (member host was a ClusterIP, which `mongod` 's `isSelf()` can never match) fixed via headless per-pod DNS.

Live ChaosCenter UI batch launches `2026-08-13 → 08-19`

36+ experiments launched through the actual web UI (not just the CLI driver) surfaced platform-level bugs: namespace stuck "Terminating," orphaned infra namespaces, a broken "copy this kubectl command" link that only worked if the browser and terminal agreed on an address, and a UI stuck on "Loading..." traced to Kubernetes' internal traffic routing being silently broken since cluster creation.

SRE agent (Zero) and SRE-CrewAI onboarded `2026-08-14 → 08-19`

Both agents wired into `setup.sh` and the UI; the 29 custom ITBench faults were rewritten from plain shell scripts into real chaos-testing-framework programs so they can actually report pass/fail instead of always looking successful.

The most recent real experiment: 29/29 faults ran, but the agent never actually looked

`~2026-08-19/20` `● root cause found`

The current benchmark of record. All 29 ITBench faults were triggered against `sre-agent-comprehensive` exactly the way clicking "Run" in the web UI triggers them — not a driver shortcut. That surfaced 4 real platform bugs (cluster-internal DNS routing silently broken; a required readiness precheck rejecting valid faults; unreliable in-cluster file copying; one stuck run silently delaying every run behind it), all fixed. Every fault injected cleanly and every run showed real LLM activity.

But every one of the 29 diagnoses came back "found nothing." The agent's very first tool-call attempt, every single run, didn't match the format its framework (CrewAI) expects — and CrewAI's fallback on a malformed first attempt is to demand a final answer immediately rather than retry. Root cause: the same issue behind the `gpt-4o` → GPT-5.1 caveat above — a reasoning-class model doesn't honor the `stop` sequence CrewAI's ReAct parser depends on. The platform and fault-injection machinery are now proven end-to-end; the agent riding on top of it was not producing usable diagnoses. See today's fixes for the candidate repair.

setup.sh hardening marathon + agent-correctness fixes `2026-08-20 → today`

`● commit status disputed`

A long tail of wizard reliability fixes (dependency audits, a silent-death job-control bug, adaptive build parallelism, Enter-key defaults), an infra manifest-ordering race fix, and — today — the candidate fix for the "found nothing" bug above, plus a related flash-agent honesty fix. See below.

04 · SIX INCIDENTS, ONE STILL OPEN

Rootless Docker: the full accounting

This checkout runs on a **personal rootless Docker daemon** (`./scripts/setup.sh --rootless-docker`, CLAUDE.md §6) instead of the shared host's root daemon — deliberately, to avoid the cross-checkout collision risk described in CLAUDE.md §0. That trade bought real isolation but also a genuine tail of incidents, since RootlessKit's private network namespace, its own containerd, and per-UID kernel resource limits behave differently from the rootful daemon in ways nothing upstream documents clearly. Six distinct incidents so far — five fixed or self-healing, one still open and worth knowing about before it bites again.

INCIDENT	WHAT ACTUALLY HAPPENED	STATUS
Bridge networking + KinD internal-kubeconfig (§20–22)	RootlessKit's "host" network is its own private netns, not the real host's — `auth` / `graphql` / `web` on `network_mode: host` came up with no error but were unreachable. Moved to bridge networking + explicit `ports:`; `graphql` needed a separate fix (an internal, container-DNS-resolvable kubeconfig) to keep reaching the KinD API server once it left the shared host netns.	not run e2e
`containerd≥2.3.0` shim regression, `"unsupported protocol: Yunix"` (§23)	This host's system containerd package (2.3.3) has a confirmed upstream bug breaking every container start under the affected daemon. Downgrading the system package would risk other users' containers on the shared root daemon, so instead a checksum-verified static containerd 2.2.6 is pinned under `$HOME`, scoped only to the personal rootless `docker.service` user unit.	self-healing
KinD node DNS unreachable via gateway IP (§47/48)	KinD rewrites each node's `/etc/resolv.conf` to the node's Docker-network gateway IP when it detects a loopback host resolver (always true here, systemd-resolved). The rootful daemon transparently proxies that gateway IP to real DNS via iptables DNAT; RootlessKit/slirp4netns does not — so every external image pull inside every node timed out, and the ChaosCenter UI just showed infra stuck "Pending" forever with no hint why.	not verified
`kind load docker-image` / `docker cp` deposit stale image content (§65 Problem 3)	This host's rootless daemon uses the `containerd-snapshotter` storage driver. A freshly-rebuilt image, verified correct on the host, was still landing on the KinD node as stale content 3× in a row — `docker cp` even reported exit 0 while the destination file provably didn't exist. Root cause in the daemon itself was never isolated.	workaround

INCIDENT	WHAT ACTUALLY HAPPENED	STATUS
Ollama Service left pointing at a dead endpoint (§59 investigation)	The `ace` namespace's Ollama Service is a manually-bound Endpoints object pointing at a fixed docker-bridge gateway port. During earlier rootless-Docker migration work, the actual Ollama container was removed (not just stopped) — the model volume survived, but flash-agent pods calling it got `Connection refused`, silently reported as a clean scan (see the flash-agent honesty fix above).	● needs restart
Per-UID kernel session-keyring exhaustion — the "disk quota exceeded" that isn't about disk (§62)	An Argo-driven experiment failed 5 of ~20 app pods with `OCI runtime create failed: unable to create session key: disk quota exceeded`. Root-caused via `/proc/key-users`: this rootless-Docker UID had hit `200/200` on `kernel.keys.maxkeys` — the host's default per-user kernel session-keyring quota, which every new container's OCI runtime consumes one of. Pressure came from an unrelated `flash-agent` release crash-looping for 28h/339 restarts (left running deliberately, on the user's own instruction) — each restart burns another key. Recovered only by deleting a throwaway namespace; no sysctl was raised and nothing monitors this quota.	● not fixed

The kernel-keyring quota is a silent, cluster-wide time bomb under rootless Docker

● unresolved

Every new container consumes one session key against a **per-Linux-user** kernel quota (`kernel.keys.maxkeys`, default 200) — this is a host/kernel limit, not a Docker or Kubernetes setting, and it is shared across every rootless daemon, every KinD cluster, and every pod that user's checkout ever creates. A crash-looping pod left running for any real length of time (as CLAUDE.md §0 and this session's own instructions correctly favor over unilaterally killing someone's in-progress work) burns through it quietly, then a completely unrelated experiment fails with an error that reads like a disk problem.

RECOMMENDATION

Two independent fixes, not mutually exclusive: (1) raise the quota for this user — `sudo sysctl -w kernel.keys.maxkeys=`, persisted via a `/etc/sysctl.d/` drop-in, host-admin coordinated since it's kernel-wide, not per-checkout; (2) add a preflight check to `setup.sh` / `check-prerequisites.sh` that reads `/proc/key-users` for the current UID and warns before it's exhausted, the same way the existing dependency audits already warn on other resource gaps.

05 · TODAY'S SESSION

What got fixed today (2026-08-25)

Two batches: fixes to agent/LLM-routing correctness (landed in commit `39ad013`), and a further run of `setup.sh` wizard-reliability fixes plus one infra bug (§75–§81 of the handoff log, disputed commit status — see the callout above).

LLM routing & agent correctness

Azure gpt-4o stopped silently dropping calls

litellm.yaml

LiteLLM v1.82's shared aiohttp connection pool can hand out a stale pooled connection to a backend that already closed it idle, surfacing as a bare `APIConnectionError` — and a single-deployment model group gets fully blackballed after 3 such failures in 10s, aborting retries regardless of `num_retries`.

FIX

Added a second, deliberately-distinct Azure gpt-4o deployment entry so failures spread across two deployment ids instead of blackballing one; set `AIOHTTP_KEEPALIVE_TIMEOUT=45` below the backend's own idle-close window; raised `num_retries` 3→5.

flash-agent stopped reporting fake health

flash_agent.py

When MCP tool discovery failed or the ReAct loop dead-ended, the agent silently returned the same shape as a genuinely healthy scan (`issues: []`) — indistinguishable from "checked, found nothing wrong."

FIX

Both failure paths now return an explicit `status: "failed"` with a reason, and `main.py` exits non-zero instead of reporting success.

sre-agent-comprehensive: candidate fix for "found nothing"

crew.py

Ported the same stop-word / `max_completion_tokens` workaround already proven in `sre-agent-crewai` (see 4.7) — the model behind `gpt-4o` rejects the legacy `stop` / `max_tokens` params CrewAI's ReAct executor sends by default, which is the most direct explanation for every diagnosis in the run above coming back empty.

STATUS

Not yet re-verified live. Shipped in the same commit as the finding, but no fresh 29-fault run has confirmed it actually produces real diagnoses — see onboarding plan, step A3.

Wizard reliability & infra — §75–§81

Venv sync flooded the terminal

§75

Installing every `requirements*.txt` in the repo into one `.venv` printed raw pip resolver/download noise, and wasn't excluding `.venv-*` directories — so it was also installing requirements files found *inside* vendored packages like `embedchain`.

FIX

Pip output now redirects to `.tmp/prereq/python-venv-sync.log`; terminal shows a compact `[####----] n/N` progress bar; `.venv` / `.venv-*` excluded from manifest discovery.

pip `resolution-too-deep`

§76

The aggregated `pyproject.toml` dependency install forced one giant transitive solve across unrelated stacks (CrewAI/chromadb pulled in `backoff` / `posthog` backtracking) and exceeded pip's resolver depth.

FIX

That step now installs with `--no-deps`; requirements manifests remain the source of truth for transitive deps.

Noisy `SyntaxWarning` spam §77

A real repo bug (unescaped regex in `chaos-charts/scripts/version/version_validator.py`) plus third-party warnings from installed packages (`pysbd`, etc.) cluttered every setup run.

FIX

Regex literal converted to a raw string at source; `PYTHONWARNINGS=ignore::SyntaxWarning` now set for setup-managed Python subprocesses.

Build prompt defaulted to skip §78

Pressing Enter at the image-build prompt used to mean "skip," which is a surprising default for a first-time setup.

FIX

Prompt now reads `[p/l/A/n]` — Enter maps to "build ALL locally" in both express and guided setup paths.

Prereq audit looked hung §79

The dependency and import audits skipped a directory literally named `.venv`, but this checkout's venv is `.venv-setup-auto` — so both audits silently crawled **32,409 files** of site-packages, producing hundreds of false positives with zero visible progress.

FIX

`check-prerequisites.sh` now skips any path containing `site-packages` or any component starting with `.venv`, catching every venv name without enumerating them.

Build phase looked hung on Ollama §80

After all build logs finished writing, the script kept waiting — a bare `wait` was blocking on *every* background job, including the multi-GB `qwen2.5:32b-instruct` Ollama pull started much earlier, which produces no terminal output for minutes.

FIX

Bare `wait` → `wait "${_build_pids[@]}"`, scoped to only the build jobs from that loop.

ITBench infra registration raced RBAC vs. its own Deployments

§81

The Go manifest generator glues 8 template files (`1a/1b` … `4a/4b` — the "a" files are RBAC, the "b" files are the Deployments needing it) using whatever order the filesystem returns, not filename order. Live evidence caught on this cluster: Kubernetes briefly failed to create `prometheus-mcp-server` because its ServiceAccount didn't exist yet.

FIX

Sort the template file list before concatenating (`AgentCert/.../infra_utils.go`). **Not yet in the running image** — needs `setup.sh --restart --local-build` to rebuild the graphql Go binary before it takes effect on a live cluster.

06 · EARLIER, REPRESENTATIVE FIXES

Bug catalog, by layer

A sample from the ~70 earlier entries — grouped by where the bug actually lived, since that's usually the more useful cut for a new developer than chronological order. Full detail and commit SHAs are in the handoff log.

LAYER	BUG	ROOT CAUSE
Cluster / infra	Litmus `subscriber` pod, 876 restarts	NetworkPolicy blocked its own ingress selectors
Cluster / infra	Auth service, 159 restarts over 17h	Mongo replica member registered by ClusterIP name, which `isSelf()` can never match — fixed via headless per-pod DNS
LLM gateway	Ollama unreachable from LiteLLM container	Ollama bound to `127.0.0.1` only
LLM gateway	Prompt silently truncated	Ollama's default 2048-token context window vs. flash-agent's ~2100-token system prompt; fixed with explicit `num_ctx: 16384`
Agent code	CISO agent crashed for any non-GPT model	Two separate LLM call paths (LangChain + CrewAI/litellm) each hardcoded a `"gpt" in model` check
Agent code	flash-agent crashed on some model outputs	`[{...}]` list-wrapped JSON parsed successfully but broke every downstream `.get()` call
Certifier	Phase 4 PDF silently dropped half its content	Renderer's dispatch table only handled 7 of 14 block types — entire Limitations/Recommendations sections rendered blank with no error
Certifier	`total_runs: 0` in every report	Key-path typo: extractor read `experiment.run_id`, real field is `experiment_run_id`
Mass-execution driver	Cluster left in broken state after full sweep	ChaosEngine deleted immediately after the agent's scan finished, before the fault's own revert commands had run

07 · AS OF THIS WRITING

What's actually unresolved right now

Everything below is either mid-flight, genuinely open, or a design gap that's been mitigated rather than solved. Nothing here is hidden or worked around — see each entry's recommendation.

Commit status is disputed — verify before trusting either claim

● unresolved

This session was told everything from today is committed. A direct `git status` in this checkout, run while writing this page, still shows the exact same uncommitted diffs as before that claim: `scripts/setup.sh`, `scripts/check-prerequisites.sh`, `scripts/sync-python-venv.sh`, both handoff docs, plus real uncommitted diffs inside the `AgentCert` submodule (the §81 Go fix) and `chaos-charts` submodule (the §77 regex fix). Not asserting the claim is wrong — it may be accurate in a different checkout on this shared host — but it does not match what this checkout shows.

RECOMMENDATION

Run `git status` (and `git -C AgentCert status` / `git -C chaos-charts status`) yourself before assuming any of today's work is landed. If this checkout is the one that matters, commit per-repo with Conventional Commits, bump submodule pointers, and push — following the pattern in §74 of the handoff log.

Every install-agent step still gets flash-agent-shaped values, regardless of which agent is selected

● blocks multi-agent onboarding

`injectExperimentContextArgs` (GraphQL server) matches any install-agent workflow step by template name alone — it never reads which agent chart is actually being installed — and unconditionally injects a fixed Helm `--set` arg list shaped for flash-agent's specific schema (`agent.config.MCP_URLS`, `agent.config.OPENAI_BASE_URL`, etc.). `sre-agent-comprehensive` already uses a structurally different schema (`agent.notifyId`, `agent.workflowUid` — no `agent.config.*` namespace at all) with no branching to handle it.

RECOMMENDATION

This is the central blocker for treating any agent besides flash-agent as reliably configured through the standard UI/Argo experiment path — see step A1 of the onboarding plan.

Per-UID kernel keyring exhaustion can silently block all new pods

● unresolved

`kernel.keys.maxkeys` (default 200, per Linux user) is shared across every rootless-Docker container this checkout ever creates; a long-running crash-looping pod quietly burns through it, then an unrelated experiment fails with a misleading "disk quota exceeded." Full incident writeup in Rootless Docker.

RECOMMENDATION

Raise the sysctl for this user (host-admin coordinated) and/or add a `/proc/key-users` preflight check to `setup.sh`.

Ollama's Service may still point at a dead endpoint

● needs a check

Found dangling after an earlier rootless-Docker migration removed the actual container while the Service/Endpoints object and model volume survived — not confirmed fixed since.

RECOMMENDATION

Verify with `docker --context rootless ps -a | grep ollama`; if absent, `./scripts/start-local-services.sh --only-ollama`.

Today's "found nothing" fix hasn't been re-verified live

● unverified

The stop-word/`max_completion_tokens` fix ported into `sre-agent-comprehensive` is a strong candidate for why every one of the 29 real UI-triggered runs came back with an empty diagnosis (see timeline), but no fresh run has confirmed it.

RECOMMENDATION

Re-run the same 29-fault sweep through the real UI-trigger path and confirm the agent's diagnoses are no longer uniformly empty — step A3 of the onboarding plan.

Cross-component Python dependency conflict — mitigated, not solved

design gap

The setup venv-sync flattens every ACE Python component into one `.venv`. That's fragile by design: `agents/ciso-agent` pins older OpenAI/LangChain/OTel/tokenizer packages while `certifier` and the SRE agents allow newer ones. Today's fixes (§75–76) made the symptom quieter and stopped one resolver failure mode — they didn't change the underlying one-venv-for-everything architecture.

RECOMMENDATION

If this resurfaces as a real install failure (not just noise), the fix is separate venvs per component, or reconciled pins — not another resolver flag.

Rootless-Docker Compose rework: parse-verified, not run end-to-end

unverified

The bridge-networking rework for `auth` / `graphql` / `web` under rootless Docker (§20 of the handoff, first row of the Rootless Docker table) was verified via `docker compose config` — i.e. the rendered YAML looks correct — but a real `docker compose up` bring-up under a rootless daemon has not actually been run.

RECOMMENDATION

Before relying on this for anything beyond local iteration, run it live: `./scripts/setup.sh --rootless-docker --restart` then `./scripts/compose-up-guard.sh up -d`, and confirm all services reach each other by service-name DNS.

`CLUSTER_MODE=cloud` + rootless Docker: known, unfixed gap

not fixed

The privatelink-DNS `pin_api_server_host()` mechanism still only patches `/etc/hosts` for the host-networked `cluster-init` service. Once `graphql` moved to bridge networking it stopped sharing that file, so this pinning never reaches it in cloud mode.

RECOMMENDATION

Doesn't block local-KinD work (the rootless rework's actual target). Tracked in `innovation.md` §3.19 — read that before touching cloud-mode DNS pinning.

Handoff doc status banners are stale

docs only

Both handoff docs' top-of-file status line still says "as of 2026-08-12, 137/137 runs" even though the log content now runs through §80 / 2026-08-25. Low-risk, but worth a pass so the next reader doesn't anchor on the wrong date.

RECOMMENDATION

Update the banner in the same commit that lands today's §75–80 entries.

Three CISO narrative builders still stub out on CISO-only runs

known gap

`key_findings`, `qualitative`, and `limitations` builders read an SRE-only field (`fault_detection_success_rate`) for every category; on CISO-only certification runs this is caught gracefully and falls back to placeholder text rather than crashing, but the CISO-specific narrative content itself is still generic.

RECOMMENDATION

Genuine follow-on scope — write real CISO-aware templates for these three builders using `ciso_task_pass_rate` instead.

08 · PRIORITIZED PLAN

What's left before ITBench faults are fully onboarded onto standard benchmarking

All 29+ ITBench SRE fault bundles exist and inject cleanly (§64/65). What's not yet true: that any onboarded agent besides flash-agent is reliably *configured* by the standard UI/Argo path, that the one agent under active test is actually *diagnosing* anything, or that a result can be trusted to say which model produced it. Ordered by what blocks the next thing.

Phase A — Correctness blockers (fix before trusting any multi-agent result)

A1 · Stop hardcoding flash-agent's value schema

Fix `injectExperimentContextArgs` to branch on the selected agent's declared schema (via `agenthub`'s per-agent metadata, the same mechanism already used for image/pull-policy resolution) instead of injecting one fixed arg list onto every install-agent step.

A2 · Rebuild + redeploy the §81 fix

The manifest-ordering fix lives in Go source but not the running image. `setup.sh --restart --local-build`, then register a fresh infra and confirm RBAC lands before its Deployments with no race.

A3 · Re-verify the "found nothing" fix

Re-run the full 29-fault sweep against `sre-agent-comprehensive` through the real UI-trigger path and confirm diagnoses are no longer uniformly empty before trusting any certification built on top of it.

A4 · Pin and report actual model identity

Add a validated model/provider field at agent registration, extract the real `model` from Langfuse generation spans into the cert report's `Meta` section. Non-optional given the confirmed gpt-4o→GPT-5.1 mislabeling — a report currently cannot say what it actually measured.

Phase B — Coverage (today, only one of six agents has current data)

B1 · Repeat the real-UI-trigger validation for the rest

`sre-agent`, `sre-agent-crewai`, `ciso-agent`, and `k8s-agent` have no recent live-UI-triggered data — everything current is `sre-agent-comprehensive` only.

B2 · Decide the fate of the 137/137 dataset

It's the only complete N-run dataset that exists, but it predates the Azure routing and the real trigger path. Either re-run it under current config or explicitly retire it as a historical baseline in the reports that reference it.

Phase C — Scale & safety (needed once running the full fault set becomes routine)

C1 · Fix the disk-fill blast-radius gap

Sizing falls through to the node's real shared disk when no `ephemeral-storage` limit is set on the target — set limits on every valid target chart before running it unattended.

C2 · Concurrent experiment support

Only one experiment can safely run per agent/app pairing today (shared names, shared NodePorts, no reservation model) — needed if the goal is running large fault×agent batches faster than serially.

C3 · Guided onboarding UX

No "Create Experiment" path exists for a new agent/app pairing — chart-folder names must match exactly by hand, and Save silently stays disabled without a fault step. Real friction for onboarding future agents.

Phase D — Housekeeping

D1 · Resolve the commit-status discrepancy

See Current problems — confirm which checkout is authoritative, commit and push there.

D2 · Update the stale status banners

Both handoff docs still open with "as of 2026-08-12" even though content runs through §81 — update in the same pass as D1.

---

09 · INNOVATION.MD DIGEST

Ideas for later

`innovation.md` is the consolidated log of every feature considered across ACE's development — roughly 51 entries already implemented, ~20 still proposed, and a few partially tested or waiting on an upstream PR. What follows is only the open ones, grouped by area — the implemented ones are either already covered above or are invisible plumbing not worth repeating here. Reference numbers match `innovation.md` section numbers directly.

IDEA	AREA	REF
Per-experiment model selection from the ChaosCenter UI, not a global `.env` setting	Certifier	1.11
Fault-injection timestamp sourced from Argo workflow state, not a trace-native event — biases TTD/TTR under node contention	Certifier	1.13
No model/provider identity control, validation, or reporting for agents under test	Certifier	1.14
LiteLLM rate-limit controls (`rpm` / `tpm` / `max_parallel_requests`) unconfigured for cloud providers	LLM Config	1.15
Switch `sre-agent-qwen` to the composite prompt entry-point — deliberately deferred to avoid invalidating capability-probe baselines	Agents	2.7
Streaming early-abort for stop-incompatible model aliases — working prototype, not the default	LLM Config	4.7
Multi-stage self-contained web Dockerfile (build frontend inside Docker, not from a pre-built `dist/`)	Infrastructure	3.14
`make dev` target for always-fresh Compose images	Dev Experience	3.15
Prompt for JWT/MongoDB credentials at setup instead of shipping defaults	Security	3.16
Prompt for Azure Content Safety / Blob Storage config at setup	Security	3.17
HTTPS for the ChaosCenter web UI — currently plaintext end to end	Infrastructure	3.18
Concurrent experiment execution for the same agent/app pairing	Infrastructure	3.20
`disk-fill` fault blast-radius audit	Infrastructure	3.21

IDEA	AREA	REF
MongoDB backup/restore/lineage flow needs a real end-to-end checkup against a live pod	Infrastructure	3.22
`--local` flag for `build-and-push.sh` (kind-load instead of Hub push)	Images	5.3
`make setup-env` target for gitignored per-agent trial env files	Dev Experience	6.7
"Answer all questions upfront" option in `setup.sh`	Dev Experience	6.8
Origin header on the GraphQL subscriber's WebSocket dial, instead of a broadened `ALLOWED_ORIGINS` regex	Security	7.4
PR for the CISO agent OpenAI-compatible LLM fix — sitting on a personal fork, not yet raised upstream	Upstream	8.1
Push the SRE agent live-mode MCP fix — committed locally, not yet pushed anywhere	Upstream	8.2
Remaining CISO scenario types untested: Kubectl-OPA, RHEL9-Ansible-OPA, Upd-Kyverno	Evaluation	8.3

10 · ORIENTATION CHEATSHEET

Quick reference

Start here

DOC	WHAT'S IN IT
`CLAUDE.md`	Authoritative repo map — architecture, subsystems, env vars, entry points. Read first.
`OPEN_WEIGHT_CERTIFICATION_HANDOFF.md`	The full 80-entry technical change log this page summarizes — commit SHAs, exact commands, verification steps for every fix.
`OPEN_WEIGHT_CERTIFICATION_HANDOFF_READABLE.md`	Prose rewrite of the same log, easier to skim.
`chaos-charts/ITBENCH_HANDOFF.md`	Design notes for the 30 ITBench fault bundles themselves (submodule).
`innovation.md`	Every feature considered or implemented across ACE, with status — source for Ideas for later.

Commands you'll actually use

COMMAND	DOES WHAT
`./scripts/setup.sh`	Interactive first-time wizard — creates `.env`, generates Helm values, optionally deploys
`./scripts/setup.sh --restart`	Redeploy with existing `.env`, no prompts (does *not* rebuild Go binaries — add `--local-build` if you changed Go code)
`./scripts/start-local-services.sh`	Bring up MongoDB + Langfuse + Ollama + LiteLLM + Certifier via Compose
`python scripts/ace-bench.py flash-agent --runs 30`	Dev-tool: run the flash-agent trace_based pipeline locally
`python scripts/run_certification.py --trace-id <UUID>`	Run the certifier pipeline for one trace, no FastAPI server needed

Default local credentials

SERVICE	URL	LOGIN
AgentCert UI	`localhost:2001`	admin / litmus
Langfuse	`localhost:$LANGFUSE_PORT` (4000)	admin@agentcert.local / agentcert-admin
MongoDB	`localhost:$MONGO_PORT` (27017, rs0)	admin / 1234
LiteLLM proxy	`localhost:$LITELLM_PORT` (14000)	key: `sk-agentcert-2026`
Certifier Swagger	`localhost:8000/docs`	—

The one rule that matters most on this host

This dev host is shared. Never stop, remove, or recreate a Docker/Kubernetes resource you didn't create this session without explicit confirmation — see `CLAUDE.md` §0. An implicit Compose project name once caused an agent session to silently delete another user's running containers. Always use `ACE_INSTANCE_NAME`-scoped naming and `scripts/compose-up-guard.sh`.

Built from `OPEN_WEIGHT_CERTIFICATION_HANDOFF.md` / `_READABLE.md` (81 entries), `innovation.md`, `CLAUDE.md`, and a direct look at git status, the live litellm/agent config, and running processes on 2026-08-25. Treat the handoff docs and `innovation.md` as the source of truth for anything condensed here — this page trims for orientation, not completeness, and corrects forward from what earlier sessions believed (see the callout at the top) rather than silently restating it.